Machine Learning Assignment 2 - Submission

M.Tech (AIML) • Machine Learning

Name: Kavin R A | BITS ID: 2026ak05552

---

1. GitHub Repository Link

https://github.com/2026ak05552-png/ML_Assignment02_2026ak05552

This repository holds the complete work for the assignment: the Streamlit application (**app.py**), **requirements.txt**, **README.md**, **test_data.csv**, and the **model/** folder with the five saved trained pipelines. I developed and committed these files across the course of this assignment.

2. Live Streamlit App Link

https://mlassignment02.streamlit.app/

The application is hosted on Streamlit Community Cloud. It opens an interactive frontend where a user can upload a test CSV, pick a model from the dropdown, and see the evaluation metrics, confusion matrix, and classification report. No retraining happens on the server — the app loads the saved pipelines directly.

3. Screenshot - BITS Virtual Lab Execution

[Insert screenshot here showing the Streamlit application running on BITS Virtual Lab]

4. README.md Content

a. Problem Statement

The goal of this project is to predict whether a bank customer will subscribe to a term deposit. This is a binary classification problem (yes or no). I used customer and campaign attributes, trained five different classification models, and packaged the results into a Streamlit web application for interactive demonstration.

b. Dataset Description

I chose the Bank Marketing dataset from the UCI Machine Learning Repository (*bank-additional-full.csv*). It contains 41,188 customer records, which is well above the minimum of 500 instances required. After removing the *duration* column to avoid target leakage, the dataset keeps 19 input features, exceeding the required minimum of 12. The target column *y* shows subscription status (*yes* = 1, *no* = 0). The data is imbalanced, with about 36,548 negative responses and 4,640 positive responses.

c. GitHub Repository Link

https://github.com/2026ak05552-png/ML_Assignment02_2026ak05552

d. Models Used

All five models were trained on the same train/test split and the same preprocessing pipeline, so the comparison is fair. The models are:
• Logistic Regression
• Decision Tree Classifier
• K-Nearest Neighbors Classifier
• Naive Bayes Classifier (Gaussian)
• Support Vector Machine (LinearSVC)
Each model was saved as a complete scikit-learn pipeline (.joblib) that bundles preprocessing and classification together, so the Streamlit app can load them directly without retraining.

Model Comparison Table

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9009	0.8008	0.6905	0.2188	0.3322	0.3516
Decision Tree	0.8418	0.6258	0.3131	0.3384	0.3252	0.2360
KNN	0.8972	0.7444	0.5839	0.3039	0.3997	0.3719
Naive Bayes	0.8049	0.7755	0.3172	0.6347	0.4230	0.3490
Support Vector Machine	0.9014	0.7979	0.7164	0.2069	0.3211	0.3502

Observations on Model Performance

ML Model Name	Observation
Logistic Regression	Delivers high accuracy (0.9009) and strong precision (0.6905) but low recall (0.2188), indicating it is conservativ
Decision Tree	Shows moderate accuracy (0.8418) but the lowest AUC (0.6258) and MCC (0.2360). Captures non-linear patter
KNN	Achieves the best balance with the highest MCC (0.3719) and strong F1 (0.3997). Distance-based performance
Naive Bayes	Posts the highest recall (0.6347), identifying most positive cases, though at the cost of lower precision (0.3172).
Support Vector Machine	Records the highest accuracy (0.9014) and precision (0.7164) but very low recall (0.2069), making it overly cons

Overall Winner

The winning model is **KNN**, selected based on the highest Matthews Correlation Coefficient (MCC = 0.3719). MCC is the most reliable metric for imbalanced classification. MCC and F1 are prioritized over accuracy because the dataset is heavily skewed toward *no* responses (36,548 vs 4,640).

5. Source Code (app.py)

Below is the Streamlit application source code. It loads the saved model pipelines, accepts a CSV upload, lets the user select a model, and displays metrics, confusion matrix, and classification report without retraining.

```
import streamlit as st
import pandas as pd
import numpy as np
import joblib
import os
import warnings

warnings.filterwarnings("ignore")

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    accuracy_score,
```

```
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    matthews_corrcoef,
    confusion_matrix,
    classification_report,
)

st.set_page_config(
    page_title="Bank Marketing Model Comparison",
    layout="wide",
    initial_sidebar_state="expanded",
)

st.markdown(
    """
    <style>
    .block-container { padding-top: 0.5rem; padding-bottom: 2rem; max-width: 1100px; }
    .app-title { font-size: 22px; font-weight: 700; color: #14273b; margin-bottom: 4px; }
    .app-subtitle { font-size: 13px; color: #6b7a8d; margin-bottom: 24px; }
    .section-heading { font-size: 17px; font-weight: 600; color: #14273b; margin-top: 28px; margin-bottom: 12px; border-bottom:
    .metric-box { background: #f5f8fb; border: 1px solid #dde5ee; border-radius: 8px; padding: 14px; margin-bottom: 10px; }
    .metric-name { font-size: 11px; color: #7a8b9d; text-transform: uppercase; letter-spacing: 0.6px; margin-bottom: 4px; }
    .metric-value { font-size: 18px; font-weight: 700; color: #14273b; }
    .small-note { font-size: 12px; color: #7a8b9d; margin-top: 6px; }
    </style>
    """,
    unsafe_allow_html=True,
)

MODEL_DIR = "model"
MODEL_FILES = {
    "Logistic Regression": "logistic_regression.joblib",
    "Decision Tree": "decision_tree.joblib",
    "KNN": "knn.joblib",
    "Naive Bayes": "naive_bayes.joblib",
    "Support Vector Machine": "support_vector_machine.joblib",
}

@st.cache_resource
def load_artifacts():
    artifacts = {}
    artifacts["models"] = {}
    for name, fname in MODEL_FILES.items():
        artifacts["models"][name] = joblib.load(os.path.join(MODEL_DIR, fname))
    try:
        artifacts["metrics_df"] = pd.read_csv(os.path.join(MODEL_DIR, "metrics_comparison.csv"))
    except Exception:
        artifacts["metrics_df"] = None
    return artifacts

try:
    artifacts = load_artifacts()
    models = artifacts["models"]
    metrics_df = artifacts["metrics_df"]
    artifacts_loaded = True
except Exception as e:
    artifacts_loaded = False
    load_error = str(e)
    import traceback
    traceback.print_exc()

st.markdown('<div class="app-title">Bank Marketing Classification</div>', unsafe_allow_html=True)
st.markdown('<div class="app-subtitle">Compare five trained models on uploaded test data. Metrics are computed from saved pipel

if not artifacts_loaded:
    st.error("Model artifacts not found. Error: " + str(load_error))
    st.info("Debug: Check Streamlit Cloud logs for the full traceback.")
    st.stop()

st.sidebar.markdown("### Controls")
uploaded_file = st.sidebar.file_uploader("Upload test CSV", type=["csv"])
selected_model_name = st.sidebar.selectbox("Select model", list(MODEL_FILES.keys()))
show_report = st.sidebar.checkbox("Show classification report", value=True)
show_matrix = st.sidebar.checkbox("Show confusion matrix", value=True)

if uploaded_file is None:
    st.info("Upload a test CSV file from the sidebar to begin. The file must contain all input features and the target column (
    st.stop()

try:
    df_raw = pd.read_csv(uploaded_file)
except Exception as e:
```

```
    st.error("Failed to read CSV: " + str(e))
    st.stop()

# Preprocess upload using each model's own pipeline (encoders + scaler bundled inside)
try:
    model = models[selected_model_name]
    y_pred = model.predict(df_raw.drop(columns=["y"], errors="ignore"))

    if hasattr(model, "predict_proba"):
        y_prob = model.predict_proba(df_raw.drop(columns=["y"], errors="ignore"))[:, 1]
    else:
        y_prob = model.decision_function(df_raw.drop(columns=["y"], errors="ignore"))

    y_true = df_raw["y"] if "y" in df_raw.columns else None
except Exception as e:
    st.error("Prediction failed: " + str(e))
    st.stop()

if y_true is not None:
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)
    try:
        auc = roc_auc_score(y_true, y_prob)
    except Exception:
        auc = float("nan")
    mcc = matthews_corrcoef(y_true, y_pred)

    st.markdown('<div class="section-heading">Model: ' + selected_model_name + '</div>', unsafe_allow_html=True)

    col1, col2, col3 = st.columns(3)
    col4, col5, col6 = st.columns(3)

    def metric_card(col, name, value):
        with col:
            st.markdown(
                '<div class="metric-box"><div class="metric-name">' + name + '</div><div class="metric-value">' + str(value) +
                unsafe_allow_html=True,
            )

    metric_card(col1, "Accuracy", f"{acc:.4f}")
    metric_card(col2, "AUC", f"{auc:.4f}" if not np.isnan(auc) else "N/A")
    metric_card(col3, "Precision", f"{prec:.4f}")
    metric_card(col4, "Recall", f"{rec:.4f}")
    metric_card(col5, "F1 Score", f"{f1:.4f}")
    metric_card(col6, "MCC", f"{mcc:.4f}")

    if show_matrix:
        st.markdown('<div class="section-heading">Confusion Matrix</div>', unsafe_allow_html=True)
        cm = confusion_matrix(y_true, y_pred)
        fig, ax = plt.subplots(figsize=(5, 4))
        sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax)
        ax.set_xlabel("Predicted")
        ax.set_ylabel("Actual")
        ax.set_xticklabels(["No", "Yes"])
        ax.set_yticklabels(["No", "Yes"])
        st.pyplot(fig)

    if show_report:
        st.markdown('<div class="section-heading">Classification Report</div>', unsafe_allow_html=True)
        report = classification_report(y_true, y_pred, target_names=["No", "Yes"], zero_division=0)
        st.text(report)
else:
    st.warning("Target column 'y' not found in uploaded file. Showing predictions only.")
    st.write("Predictions:", y_pred[:20])

st.markdown('<div class="section-heading">All Models - Precomputed Test Metrics</div>', unsafe_allow_html=True)
if metrics_df is not None:
    st.dataframe(metrics_df, use_container_width=True)
else:
    st.warning("metrics_comparison.csv not found in model/ folder.")

st.markdown('<div class="small-note">Metrics above are calculated on your uploaded test data using the saved model pipeline. Th
```